

F24-D-117-AiUML

Project Team

Laiba Asif	21I-2560
Rabail Nasir	21I-0880
Vaneeza Ahmad	21I-0390

Session 2021-2025

Supervised by

Ms. Noor ul Ain

Co-Supervised by

Dr. Javaria Imtiaz



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Features	2
1.4	Modules	4
1.4.1	Module I: Dataset Collection and Preparation	4
1.4.2	Module II: Fine-Tuning LLM	4
1.4.3	Module III: Requirements Pre-processing	4
1.4.4	Module IV: Requirements Disambiguation (with Pre-trained Models)	5
1.4.5	Module VI: DSL Code Generation and Diagram Rendering	5
1.4.6	Module VII: Diagram Optimization with AI	5
1.4.7	Module VIII: Diagram Customizer and Editor	5
1.4.8	Module IX: Session and Diagram Code Storage	5
1.4.9	Module X: User Interface	6
1.5	User Classes and Characteristics	6
1.6	Literature Review	8
2	Project Requirements	11
2.1	Use-case Diagram	11
2.2	High Level Use Cases	12
2.2.1	UC-1:	12
2.2.2	UC-2:	12
2.2.3	UC-2.1:	12
2.2.4	UC-3:	12
2.2.5	UC-4:	13
2.2.6	UC-5:	13
2.2.7	UC-6:	13
2.2.8	UC-7:	13
2.3	Expanded Use Cases	14
2.3.1	UC - 1	14
2.3.2	UC - 2	16

2.3.3	UC - 2.1	17
2.3.4	UC - 3	19
2.3.5	UC - 4	21
2.3.6	UC - 5	23
2.3.7	UC - 6	25
2.3.8	UC - 7	27
2.4	Functional Requirements	29
2.4.1	Unrestrictive Textual Requirements Input	29
2.4.2	Automated Diagram Generation	29
2.4.3	AI Driven Diagram Optimization	30
2.4.4	Diagram Customizer and Editor	30
2.4.5	Multi Diagram Support	30
2.4.6	Exporting Diagrams in Multiple Formats	31
2.4.7	Session and Diagram Code Storage	31
2.4.8	User Friendly Interface	31
2.5	Non-Functional Requirements	31
2.5.1	Reliability	32
2.5.2	Usability	32
2.5.3	Performance	32
2.5.4	Security	32
2.6	Operation Contracts	33
2.6.1	Contract C-1:	33
2.6.2	Contract C-2:	33
2.6.3	Contract C-2.1:	33
2.6.4	Contract C-3:	34
2.6.5	Contract C-4:	34
2.6.6	Contract C-5	35
2.6.7	Contract C-6:	35
2.6.8	Contract C-7:	35
2.7	Domain Model	36
3	System Overview	37
3.1	Architectural Design	37
3.2	Architecture Diagram	38
3.3	Activity Diagrams	38
3.3.1	AD -1: Input Textual Requirements	39
3.3.2	AD -2: Generate Diagram	40
3.3.3	AD -3: Optimize Diagram with AI	41
3.3.4	AD -4: Edit Diagram in DSL Editor	42
3.3.5	AD -5: Customize Diagram with Themes	43

3.3.6	AD -6: Export Diagram in Multiple Formats	44
3.3.7	AD -7: Session and Diagram Code Storage	45
3.4	State Transition Diagram	46
3.5	Data Flow Diagram	47
3.5.0.1	Processes (Blue Color):	47
3.5.0.2	Data Stores (Gray Color):	49
3.5.0.3	External Entities:	50
3.6	System Sequence Diagrams	50
3.6.1	SSD -1: Input Textual Requirements	50
3.6.2	SSD -2: Generate Diagram	51
3.6.3	SSD -3: Optimize Diagram with AI	51
3.6.4	SSD -4: Edit Diagram in DSL Editor	52
3.6.5	SSD -5: Customize Diagram with Themes	52
3.6.6	SSD -6: Export Diagram in multiple formats	53
3.6.7	SSD -7: Session and Diagram Code Storage	53
3.7	Data Design	54
3.7.1	Key Entities:	54
3.7.2	Storage and Processing:	54
3.8	Entity Relationship Diagram	55
References		57

List of Figures

2.1	Use case diagram	11
2.2	Domain Model	36
3.1	Architecture Diagram	38
3.2	AD1 Input Textual Requirements	39
3.3	AD2 Generate Diagram	40
3.4	AD3 Optimize Diagram with AI	41
3.5	AD4 Edit Diagram in DSL Editor	42
3.6	AD5 Customize Diagram with Themes	43
3.7	AD6 Export Diagram in Multiple Formats	44
3.8	AD7 Session & Diagram Code Storage	45
3.9	State Transition Diagram	46
3.10	Data Flow Diagram	47
3.11	SSD1 Input Textual Requirements	50
3.12	SSD2 Generate Diagram	51
3.13	SSD3 Optimize Diagram with AI	51
3.14	SSD4 Edit Diagram in DSL Editor	52
3.15	SSD5 Customize Diagram with Themes	52
3.16	SSD6 Export Diagram in Multiple Formats	53
3.17	SSD7 Session and Diagram Code Storage	53
3.18	Entity Relationship Diagram (UML notation)	55

List of Tables

1.1	User classes and characteristics	7
1.2	Literature Review of Recent Articles (2023-2024)	8
1.3	Literature Review of Older Articles (2021-2022)	9
2.1	Expanded UC1	15
2.2	Expanded UC2	17
2.3	Expanded UC2.1	19
2.4	Expanded UC3	21
2.5	Expanded UC4	23
2.6	Expanded UC5	25
2.7	Expanded UC6	27
2.8	Expanded UC7	29

Chapter 1

Introduction

AiUML is an artificially intelligent system intended for automated generation of UML diagrams. By using Large Language Models, AiUML can directly extract UML diagrams from textual requirements in natural language. This will support software development in object-oriented systems and will provide clear UML diagrams for the system. AiUML aims to improve the software design workflow. It will provide prompt visual representation of systems to developers and software engineers. Thus, this will improve the efficiency and quality of software development artifacts and automate a major part of the SDLC (Software Development Life-cycle).

This paper contains the specifications for the AiUML system. Developers, project managers, marketers, users, testers, and documentation writers are the target audience for this document.

1.1 Problem Statement

There exist many inefficiencies and challenges in the manual creation of UML diagrams in software development, which is often time-consuming, prone to human error, and lacks standardization. These problems delay and disrupt project timelines, especially in complex projects. Existing tools in the market also lack as they require restrictive textual inputs and significant user intervention for diagram generation. Additionally, they are unable to extract whole UML elements, such as relationships and operations. AiUML is an intelligent solution to tackle all of these problems. It automatically generates UML diagrams directly from natural language textual requirements by using Large Language Models. These LLMs take textual software requirements as is, unlike linguistic approaches which adapt the initial text to a specific template[1]. Consequently, it recognizes UML components and generates corresponding DSL code to render UML diagrams.

1.2 Scope

The automation of UML diagram creation from natural language software requirements is the main goal of this project. It is pertinent to academic learning, software development, software engineering automation, and system design. It will cover the generation of PlantUML diagrams by utilizing natural language processing and program synthesis techniques. The system is intended for use by software engineers, system architects, project managers, and the academic community, and will support any input software requirements and high-level specification without modification. The scope includes the development, testing, and deployment of the AiUML system, to integrate it into existing software development workflows and educational environments.

1.3 Features

Following is the feature breakdown for the AiUML web application:

2.2.1 Unrestrictive Textual Input

- Allow users to input textual requirements in any format.
- Cater software descriptions, case-studies, software specifications, user stories, and functional requirements as input.
- Detect and resolve ambiguous requirements (anaphoric ambiguity).

2.2.2 Automated Diagram Generation

- UML diagrams are automatically generated from natural language text.
- Render diagrams on the screen with zoom in, zoom out options.

2.2.3 AI Driven Diagram Optimization

- Well structured and easy to understand diagrams according to software standards.
- Model complex relationships easily and produce accurate diagrams.
- Apply incremental changes to the generated diagram, and handle iterative modification requests by retaining context.

2.2.4 Diagram Customizer and Editor

- Edit diagram codes in DSL editor.
- Customize diagrams with different colors and themes.

2.2.5 Multi Diagram Support

- Support for a wide variety of diagram kinds.
- Class diagrams
- Use case diagrams
- Sequence diagrams
- Entity Relationship Diagram.

2.2.6 Exporting Diagrams in Multiple Formats

- Give multiple export options in SVG, PNG etc.
- Allow users to download diagrams easily.

2.2.7 Session and Diagram Code Storage

- Store the previous sessions of the user.
- Maintain work-spaces by saving previous diagrams and storing their codes in PlantUML format.

2.2.8 User Friendly Interface

- Create a complete Web Application for all modules.
- Simple interface for inputting textual requirements.
- Incredibly straight forward, no need to navigate through complicated menus or learn new software. Just type text description, and the AI does the rest.

1.4 Modules

AiUML is composed of several key modules, each responsible for different aspects of the UML diagram generation process. The approach being followed by our system is based on Generative AI. The modules are as follows:

1.4.1 Module I: Dataset Collection and Preparation

This module is responsible for gathering and preparing datasets necessary for training our system. It involves collecting different software requirements text corpora, annotating and labelling it to ensure that the data is formatted for large language models, to make sure that it covers a broad spectrum of software requirements.

1.4.2 Module II: Fine-Tuning LLM

This module consists of fine-tuning a text-to-code Large Language Model called Code T5. The generative model is trained on an extensive dataset and the weights are adjusted, such that it correctly generates code for a diagram when given natural language text as input.

1.4.3 Module III: Requirements Pre-processing

The Requirements Pre-processing Module is the first step, responsible for processing raw textual software requirements. It uses models to prepare the text for deeper analysis by the subsequent layers. Key functions include:

- **Text Normalization:** The module makes sure the text is standardized and clean enough to be processed. This involves doing things like resolving frequent spelling variants, eliminating extraneous punctuation, and changing text to lowercase.
- **Tokenization:** Tokens, sometimes referred to as words or subwords, are the tiny pieces of the text that the language models will process. This phase is essential for the subsequent steps that establish each token's function in the UML diagram.

This preparation allows the system to handle natural language inputs without requiring any manual adjustments.

1.4.4 Module IV: Requirements Disambiguation (with Pre-trained Models)

In this module, the processed text is passed through a hybrid ML and spanBERT solution. The ML model detects the ambiguity in requirements, specifically anaphoric ambiguity. The spanBERT model resolves the detected ambiguity. This step is crucial for subsequent processing.

1.4.5 Module VI: DSL Code Generation and Diagram Rendering

This module is responsible for generating DSL code in PlantUML or Mermaid.js from the Code T5 LLM. It ensures that the UML elements and their relationships are correctly represented in code form. After generating the code, this module uses PlantUML library or Mermaid.js library to interpret and render the final UML diagrams. By the end of this module, there will be a visual representation of the system's architecture, which users can customize or export in SVG, PNG, and text formats.

1.4.6 Module VII: Diagram Optimization with AI

This module is responsible for optimizing the diagram using a conversational LLM and CodeT5 in conjunction. After initial diagram generation, this module allows user to input feedback on the generated diagram and helps optimize diagram through prompts, like adding/removing elements, optimizing relationships according to standards or changing the layout of diagram.

1.4.7 Module VIII: Diagram Customizer and Editor

This module provides users with the ability to edit diagram codes directly within a DSL editor. It also allows for the customization of diagrams by applying different colors and themes, giving users greater control over the visual appearance and style of their UML diagrams. This functionality enhances the user experience by offering flexibility in design and presentation.

1.4.8 Module IX: Session and Diagram Code Storage

This module is responsible for storing the previous sessions of the user, ensuring that workspaces are maintained effectively. It saves previous diagrams and stores their corresponding codes in formats such as PlantUML or Mermaid.js. This functionality allows

users to revisit and edit their previous work, promoting continuity and ease of access in ongoing projects.

1.4.9 Module X: User Interface

This module involves the development of a web application that provides an intuitive interface for users. It allows users to input textual software requirements and interact with the generated UML diagrams in real-time. The interface supports features such as exporting diagrams, editing them, and customizing their appearance. Additionally, this module integrates with the backend services to store user sessions, retrieve previously saved diagrams, and manage diagram code storage in formats like PlantUML.

1.5 User Classes and Characteristics

The AiUML system is intended for a broad spectrum of users with different responsibilities in the software development life cycle and skill levels. The recognized user classes and relevant attributes are shown below:

User Class	Role	Pertinent Characteristics
Software Engineers	Software developers working on designing and implementing object-oriented systems.	1. Proficient in software development and familiar with UML standards. 2. Need quick, reliable UML diagram generation to assist in software design. 3. May require editing or customizing generated diagrams to align with specific development needs.
System Architects	Professionals responsible for high-level system design and architecture.	1. Involved in defining system structure, components, and relationships. 2. Need well-structured diagrams to represent complex relationships and architecture. 3. Use AiUML to generate initial system architecture diagrams and iterate based on requirements.
Project Managers	Individuals managing software development projects.	1. Need high-level overviews of system designs for tracking progress. 2. Require simple, understandable visual representations to communicate with teams. 3. Use AiUML to ensure design processes align with project timelines.

User Class	Role	Pertinent Characteristics
Testers/QA Engineers	Quality assurance engineers ensuring software functionality and reliability.	1. Need UML diagrams to understand system structure for developing test cases. 2. Use generated diagrams to validate system designs against requirements. 3. Require diagrams that are accurate and reflective of system operations.
Students & Educators (Academic Community)	Students learning software engineering concepts and educators teaching object-oriented design.	1. Need tools to understand and create UML diagrams as part of the learning process. 2. Require a user-friendly interface for ease of use. 3. May use AiUML to experiment with system designs.
Documentation Writers	Individuals responsible for creating and maintaining technical documentation.	1. Need clear, concise visual diagrams to complement written documentation. 2. Use AiUML to generate accurate diagrams aligned with system descriptions. 3. Need to be able to export diagrams in different file formats so that they may be included into papers.
Business Analysts	Professionals responsible for defining business needs and translating them into software requirements.	1. May not be technical experts but require visual representations of system functionality. 2. Use AiUML to visualize software requirements for stakeholders. 3. Need straightforward, easy-to-understand diagrams for communication.

Table 1.1: User classes and characteristics

1.6 Literature Review

This section provides the literature review for our project.

Article Name	Date	Technique Used	Benefits	Challenges
From text-based system specifications to UML diagrams: A bridge between words and models[3]	August 6, 2024	Linguistic Approach NLP based POS Tagging, Named Entity Recognition	Accurate extraction of classes, attributes and actors	Lower performance in relation type modeling.
Towards an Automatic UML Class Diagram from System's Textual Specifications[4]	August 12, 2024	Deep Learning with Pre-trained Language Models (LLaMA-2, GPT-2, BERT), Bidirectional LSTM Networks	Handles natural language text flexibly, Simultaneous extraction of UML diagram elements and relationships	Dataset required, approach in its beginning
Automated Software Architecture Diagram Generator using Natural Language Processing[6]	May 2023	Custom Named Entity Recognition (NER) Model using RoBERTa-base, Knowledge Graph, Masked BERT for relation extraction	Automatic generation of architecture diagrams, Knowledge graph utilization for efficient querying	Limited diagram customization, restricted relationship classification
Zero-shot Learning for NER in SRS Documents[5]	2023	Zero-shot Learning, Template-Based and Question-Answering NER methods	High accuracy in entity recognition with limited labeled data, Achieves up to 98% accuracy using QA methods	Challenges in handling insufficient labeled data, Lack of comprehensive models for specific types of software information
Deep Learning for UML Class Diagram Extraction: A Comparative Analysis[2]	2024	Deep Learning Techniques with Token Classification, Co-reference Detection, and Relationship Classification (LLaMA, GPT-2, BERT)	Reduces manual effort, improves accuracy and scalability in UML diagram generation	Challenges with natural language variability and dependency on pre-trained models

Table 1.2: Literature Review of Recent Articles (2023-2024)

Article Name	Date	Technique Used	Benefits	Challenges
Towards Automatically Extracting Class Diagrams from NL Specifications [7]	October 2022	Machine Learning and Rule-based Techniques, Segmentation and Classification of Sentences	Benchmark dataset for future research, Pattern-based diagram generation	Low precision and recall, Practical applications limited due to low accuracy
Automatic generation of UML diagrams from scenario-based user requirements [1]	June 2021	NLP Algorithm for Scenario-Based Requirements, Stanford CoreNLP for POS tagging	Automates sequence and class diagrams, Focus on detailed user scenarios	Invalid POS tagging in some scenarios, Challenges in handling less restrictive text
NLP for Systems Engineering: Automatic generation of SysML Language Diagrams [8]	November 2022	Natural Language Processing (NLP) Techniques, including Tokenization, POS Tagging, and Open Information Extraction (OpenIE)	Reduces manual effort, improves consistency and scalability in generating SysML diagrams	Struggles with highly variable text, lower precision and recall in complex relationships

Table 1.3: Literature Review of Older Articles (2021-2022)

Chapter 2

Project Requirements

The project's functional and non-functional needs are covered in this chapter.

2.1 Use-case Diagram

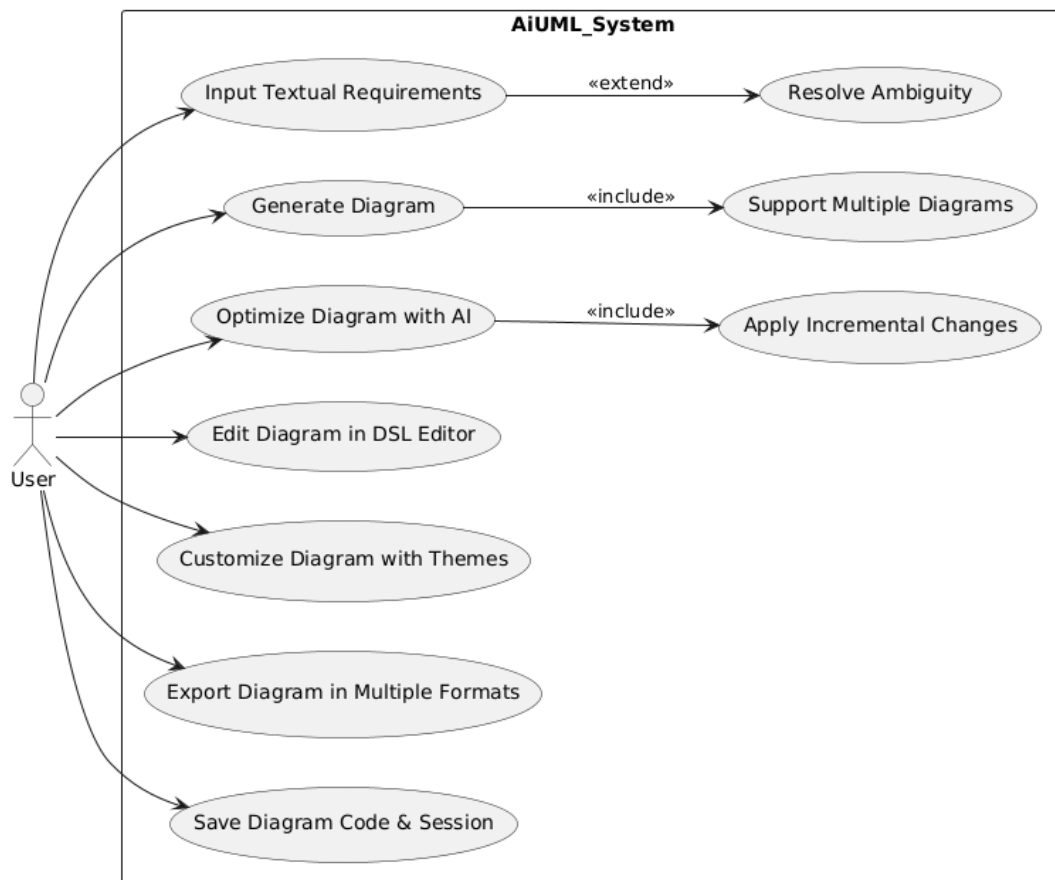


Figure 2.1: Use case diagram

2.2 High Level Use Cases

2.2.1 UC-1:

Use case: Input Textual Requirements

Actors: User

Type: Primary

Description: The user inputs natural language textual requirements into the system. The system processes and stores the input for further system operations. Any ambiguities in the input are identified and resolved.

2.2.2 UC-2:

Use case: Generate Diagram

Actors: User

Type: Primary

Description: The user provides textual requirements, and the system generates a UML diagram based on the input. The generated diagram is displayed for the user to review.

2.2.3 UC-2.1:

Use case: Support Multiple Diagrams

Actors: User

Type: Primary

Description: The user requests to generate a specific type of diagram (e.g., Class, Use Case, Sequence, or ERD). The system processes the request and generates the selected diagram type based on the textual input.

2.2.4 UC-3:

Use case: Optimize Diagram with AI

Actors: User

Type: Primary

Description: The user applies AI-driven optimization to improve the layout, structure, and clarity of the generated UML diagram for better understanding or compliance with design standards.

2.2.5 UC-4:

Use case: Edit Diagram in DSL Editor

Actors: User

Type: Primary

Description: The user modifies the diagram's code directly using a DSL editor to update its structure and components.

2.2.6 UC-5:

Use case: Customize Diagram with Themes

Actors: User

Type: Primary

Description: The user customizes the appearance of the diagram by applying themes and changing colors for better visualization and readability.

2.2.7 UC-6:

Use case: Export Diagram in Multiple Formats

Actors: User

Type: Primary

Description: The user exports a generated diagram in various formats, such as SVG or PNG, for further use or sharing.

2.2.8 UC-7:

Use case: Save Diagram Code and Session

Actors: User

Type: Primary

Description: The system allows the user to save the current diagram code and session data for future retrieval and modification.

2.3 Expanded Use Cases

The expanded use cases discuss the flow of each use case and its success scenario. Following are the expanded use cases for the use cases mentioned in the use case diagram:

2.3.1 UC - 1

Use Case ID:	UC-1
Use Case Name:	Input Textual Requirements
Actors:	User (e.g., Software Engineer, Project Manager)
Description:	The user inputs natural language textual requirements in an unrestrictive format for further system processing.
Trigger:	The user wants to provide software requirements or specifications in a freeform text format.
Preconditions:	• There is a user account on the system. • The system is operational and ready to accept input.
Postconditions:	• Textual requirements are successfully processed and stored. • Any ambiguities in the textual requirements are identified and resolved.
Normal Flow:	Actor Action: 1. The user navigates to the input interface. 2. The user inputs natural language textual requirements.

	System Response: 1. The system displays the input interface. 2. The system accepts the input from user. 3. The system processes the input and validates the format. 4. The system detects ambiguities, if any. 5. The system resolves the ambiguities and passes the result to the next module. 6. The system confirms the input has been processed.
Alternative Flows:	The user does not input valid text or enters incomplete text, and the system prompts for corrections.
Exceptions:	The system is unable to process the input due to connectivity issues or unrecognized text formats.
Frequency of Use:	Multiple times during the software development process, whenever new textual requirements are introduced.
Specific Requirements:	• The system must support unrestrictive input formats. (like software specifications, functional requirements, system descriptions, user stories). • The system must support natural language processing to identify ambiguities.
Technology and Data Variations List:	• NLP models to process the input. • Pre-built hybrid ML and SpanBERT solution for detecting and resolving ambiguities.

Table 2.1: Expanded UC1

2.3.2 UC - 2

Use Case ID:	UC-2
Use Case Name:	Generate Diagram
Actors:	User (e.g., Software Engineer, System Architect)
Description:	The user generates UML diagrams based on natural language textual input.
Trigger:	The user wants to visualize the system design or specifications as a UML diagram.
Preconditions:	• The user has inputted valid textual requirements. • The system has processed the input text completely.
Postconditions:	• The system successfully generates the diagram. • The diagram is displayed on the screen.
Normal Flow:	Actor Action: 1. The choice to create a diagram is made by the user. 2. The user waits for the system to process and display the generated diagram.
	System Response: 1. The system shows a processing status to the user. 2. The system generates PlantUML code for the diagram. 3. The system renders the UML diagram based on the generated code. 4. The system displays the generated diagram on screen. 5. The system provides zoom in and zoom out options for the displayed diagram.

Alternative Flows:	The user requests an un-supported diagram type, and the system prompts the user to choose from supported UML diagram types.
Exceptions:	The system is unable to generate the diagram due to processing errors or connection issues.
Frequency of Use:	Repeated whenever a new diagram needs to be generated based on user input.
Specific Requirements:	• The system must generate accurate diagrams based on the input. • The system must support zooming functionality for the generated diagrams.
Technology and Data Variations List:	• PlantUML • Fine tuned Code T5 LLM

Table 2.2: Expanded UC2

2.3.3 UC - 2.1

Use Case ID:	UC-2.1
Use Case Name:	Support Multiple Diagrams
Actors:	User (e.g., Software Engineer, System Architect)
Description:	Based on inputted textual requirements, the system enables the user to produce several diagram kinds, including class, use case, sequence, and entity relationship diagrams.
Trigger:	The user requests to generate a specific type of diagram based on their needs.
Preconditions:	• The user has provided valid textual requirements. • The system supports the generation of multiple diagram types.

Postconditions:	• The system successfully generates the selected diagram type (e.g., Class, Use Case, Sequence, or ERD). • The diagram is displayed on the screen for review and further modifications.
Normal Flow:	Actor Action: 1. The user selects the type of diagram they wish to generate (Class, Use Case, Sequence, or ERD). 2. The user confirms the diagram generation based on the inputted textual requirements.
	System Response: 1. The system processes the textual input and identifies the necessary components to generate the selected diagram. 2. The system generates the selected diagram type (e.g., Class, Use Case, Sequence, or ERD). 3. The system displays the generated diagram on the screen with options for further modification or export.
Alternative Flows:	The user inputs incomplete or invalid textual requirements, and the system prompts for corrections.
Exceptions:	The system fails to generate the diagram due to processing errors or unsupported diagram types.
Frequency of Use:	Whenever the user needs to generate a specific type of diagram based on textual requirements.

Specific Requirements:	• The system must support the generation of multiple diagram types (Class, Use Case, Sequence, ERD). • The system must ensure accuracy in interpreting textual requirements and mapping them to the appropriate diagram components.
Technology and Data Variations List:	• PlantUML • Code T5 LLM • Four text-to-code datasets, 1 per diagram (1000+ examples each)

Table 2.3: Expanded UC2.1

2.3.4 UC - 3

Use Case ID:	UC-3
Use Case Name:	Optimize Diagram with AI
Actors:	User (e.g., Software Engineer, System Architect)
Description:	The user applies AI-driven optimization to improve the structure and clarity of the generated UML diagram.
Trigger:	The user wants to optimize the layout (formatting, text etc), structure or components of a diagram for better understanding or compliance with design standards.
Preconditions:	• The user has already generated a diagram. • The system has access to the previous diagram context.

Postconditions:	• The diagram is optimized according to software design standards. • Any changes are applied incrementally without losing previous modifications.
Normal Flow:	Actor Action: 1. The user selects the "Optimize Diagram with AI" option. 2. The user may provide specific feedback or constraints for optimization (e.g., add/remove relationships, change font, better component placement). 3. The user waits for the system to apply the AI-driven optimizations.
	System Response: 1. The system analyzes the current diagram and identifies potential improvements in structure and relationships. 2. The system applies incremental changes using AI models to the diagram based on the user request while retaining the context of previous modifications. 3. The system updates the diagram on the screen with the optimizations. 4. The system allows the user to review the changes and make additional iterative modifications if necessary.
Alternative Flows:	The user cancels the optimization process, and the diagram remains unchanged.
Exceptions:	The system is unable to apply optimizations due to an error in the AI model or diagram complexity exceeds system limits.

Frequency of Use:	Used after generating a diagram, especially when clarity or structure improvements are needed.
Specific Requirements:	• The system must optimize diagrams according to standard design practices and user request. • The system must support incremental changes and iterative modifications.
Technology and Data Variations List:	• Code T5 LLM • Creole • Plant UML • Four Follow-up Requests Datasets, 1 per diagram

Table 2.4: Expanded UC3

2.3.5 UC - 4

Use Case ID:	UC-4
Use Case Name:	Edit Diagram in DSL Editor
Actors:	User (e.g., Software Engineer, System Architect)
Description:	The user modifies the diagram's code directly through the Domain-Specific Language (DSL) editor to update its structure and components.
Trigger:	The user wants to manually edit or modify a generated diagram's code to refine its structure or add specific components.
Preconditions:	• The user has access to a generated diagram and the DSL editor. • The user is familiar with the DSL syntax (e.g., PlantUML).

Postconditions:	• The modified diagram is updated in real-time based on the changes made in the DSL editor. • The changes are saved for future use or export.
Normal Flow:	Actor Action: 1. The user selects the option to open the DSL editor. 2. The user views the code for the generated diagram in the DSL editor. 3. The user makes manual edits to the diagram code (e.g., adding new components, modifying relationships). 4. The user saves the changes in the DSL editor.
	System Response: 1. The system opens the DSL editor with the current diagram code. 2. The system allows the user to modify the diagram code. 3. The system renders the updated diagram in real-time as the user makes changes. 4. The system saves the changes and updates the diagram accordingly.
Alternative Flows:	The user decides not to save the changes, and the diagram remains in its original form.
Exceptions:	The system fails to apply the changes due to syntax errors or unsupported modifications in the DSL code.
Frequency of Use:	Whenever the user wants to manually modify a diagram using the DSL editor.

Specific Requirements:	• The system must support DSL-based diagram editing (e.g., PlantUML). • The system must provide real-time diagram rendering based on the code changes.
Technology and Data Variations List:	• DSL (PlantUML) • HTML, CSS, Javascript • PlantUML Editor

Table 2.5: Expanded UC4

2.3.6 UC - 5

Use Case ID:	UC-5
Use Case Name:	Customize Diagram with Themes
Actors:	User (e.g., Software Engineer, System Architect)
Description:	The user customizes the appearance of the diagram by applying themes and changing colors for better visualization and readability.
Trigger:	The user wants to enhance the visual presentation of the diagram by applying a custom theme or color palette.
Preconditions:	• The user has a generated diagram available for customization. • The system provides available themes and color options.

Postconditions:	• The diagram is customized according to the selected theme or color options. • The customized diagram is rendered in real-time and saved for future use or export.
Normal Flow:	Actor Action: 1. The user opens the customization options for the diagram. 2. The user selects a theme or customizes colors for the diagram elements. 3. The user previews the changes in real-time as they are applied. 4. The user confirms and saves the customization.
	System Response: 1. The system displays the available themes and customization options. 2. The system applies the selected theme and renders the updated diagram in real-time. 3. The system updates the visual elements of the diagram based on the selected theme or color changes. 4. The system saves the customized diagram and allows the user to export it if needed.
Alternative Flows:	The user reverts the changes and the diagram returns to its original state.
Exceptions:	The system fails to apply the selected theme due to an unsupported format or error in the theme data.
Frequency of Use:	Whenever the user wishes to apply a new theme or customize the appearance of the diagram.

Specific Requirements:	• The system must support multiple themes and customizable color palettes. • The system must provide real-time rendering of theme changes.
Technology and Data Variations List:	• Puml Themes (Theme Engine) • Javascript

Table 2.6: Expanded UC5

2.3.7 UC - 6

Use Case ID:	UC-6
Use Case Name:	Export Diagram in Multiple Formats
Actors:	User (e.g., Software Engineer, System Architect)
Description:	The user exports a generated diagram in various formats, such as SVG or PNG, for further use or sharing.
Trigger:	The user wants to save or share the generated diagram in different file formats.
Preconditions:	• The user has successfully generated a diagram. • The system provides export options for multiple formats, including SVG and PNG.
Postconditions:	• The diagram is successfully exported in the selected format (e.g., SVG or PNG). • The user is able to download the exported diagram to their local system.

Normal Flow:	Actor Action: 1. The user selects the "Export Diagram" option from the interface. 2. The user chooses the desired export format (e.g., SVG, PNG). 3. The user confirms the export request. 4. The user downloads the exported diagram file.
	System Response: 1. The system displays the export options (e.g., SVG, PNG). 2. The system processes the diagram to generate the selected format. 3. The system provides the download link or prompts the user to save the exported file.
Alternative Flows:	The user cancels the export, and the diagram remains undownloaded.
Exceptions:	The system fails to export the diagram due to format incompatibility or processing errors.
Frequency of Use:	At any time the user desires to share or store the diagram in a certain format.
Specific Requirements:	• The system must support multiple export formats, including SVG and PNG. Extend to Javascript or ASCII art formats as well. • The system must export diagrams in high-quality, scalable formats (e.g., SVG).

Technology and Data Variations List:	• Export Library • PlantUML • Javascript
---	--

Table 2.7: Expanded UC6

2.3.8 UC - 7

Use Case ID:	UC-7
Use Case Name:	Save Diagram Code and Session
Actors:	User (e.g., Software Engineer, System Architect)
Description:	The system allows the user to save the current diagram code and session data for future retrieval and modification.
Trigger:	The user wants to save their current diagram and workspace progress for later use or further modifications.
Preconditions:	• There is a user account on the system. • The user has created or edited a diagram in the current session.
Postconditions:	• When the user checks in later, the diagram code will be available to them. It is preserved in their session. • The user's workspace and session data are stored for future access.

Normal Flow:	Actor Action: 1. The user selects "Logout", and sees a <i>Save</i> pop-up. (OR) 2. The user selects the "Save Diagram and Session" option. 3. The user confirms the save action.
	System Response: 1. The system prompts user to save diagram code and session before logging out. 2. The system stores the current diagram code in the specified format (e.g., PlantUML). 3. The system saves the session data, including any workspace settings and modifications made to the diagram. 4. The system confirms that the session and diagram have been successfully saved.
Alternative Flows:	The user cancels the save process, and the session remains unsaved.
Exceptions:	The system fails to save the diagram or session due to server errors or connectivity issues.
Frequency of Use:	Whenever the user needs to save their current session and diagram progress for future retrieval or modifications.
Specific Requirements:	• The system must save the diagram code in a supported format, such as PlantUML. • The system must store session data and workspace settings for future access.

Technology and Data Variations List:	• MongoDB • Firebase Real-time (alternative)
---	---

Table 2.8: Expanded UC7

2.4 Functional Requirements

The system's functional needs are explained in this section using natural language. The modules are arranged according to the name of the system feature and the particular functional needs that go along with it. The functional specifications for each feature are as follows:

2.4.1 Unrestrictive Textual Requirements Input

1. Users will be able to enter text requirements into the system in any manner that uses natural language.
2. User stories, functional requirements, case studies, software specifications, and software descriptions must all be supported by the system.
3. The system shall identify ambiguous textual requirements during input processing.
4. The system shall detect anaphoric ambiguities (e.g., unclear pronoun references) within the textual requirements.
5. The system shall provide mechanisms to resolve detected ambiguities, ensuring clarity and accuracy in the generated output.

2.4.2 Automated Diagram Generation

6. The system shall generate code for diagrams based on natural language textual input.
7. The system shall automatically render diagrams on the screen after generating the code.
8. The system shall provide options to zoom in and zoom out for the rendered diagrams on the screen.

2.4.3 AI Driven Diagram Optimization

9. The system shall generate well-structured and easy-to-understand diagrams according to standard software design practices.
10. The system shall model complex relationships between components and produce accurate UML diagrams.
11. The system shall apply incremental changes to the generated diagram based on user input while retaining the context of previous modifications.
12. The system shall handle iterative modification requests by preserving the structure and context of the diagram during multiple updates.

2.4.4 Diagram Customizer and Editor

13. Users will be able to change the diagram code directly in a Domain-Specific Language (DSL) editor thanks to the system.
14. The system will allow users to alter the diagrams' elements and structure using the DSL editor.
15. The system shall provide customization options for diagrams, allowing users to apply different colors.
16. Users will be able to choose and apply different themes to the created diagrams using the system.
17. As modifications are performed in the DSL editor, the system will instantly produce the modified diagrams.

2.4.5 Multi Diagram Support

18. The system should be able to generate a wide variety of diagram kinds.
19. Users will be able to generate **Class Diagrams** using the system by entering textual requirements.
20. Using inputted textual requirements, the system will enable users to build **Use Case Diagrams**.
21. Using inputted textual requirements, the system will enable users to generate **Sequence Diagrams**.

22. The item With inputted textual requirements, the system will enable users to build **Entity Relationship Diagrams (ERD)**.

2.4.6 Exporting Diagrams in Multiple Formats

23. The system shall provide multiple export options, including but not limited to SVG and PNG formats.
24. The system shall export diagrams in high-quality scalable vector graphics (SVG) format.
25. The system shall allow users to easily download exported diagrams.

2.4.7 Session and Diagram Code Storage

26. The system shall store the user's previous sessions.
27. The system shall save previous diagrams and their corresponding code.
28. The system shall store diagram codes in PlantUML format.
29. The system shall maintain user workspaces by storing session data for future access.
30. Users will be able to access and edit previously stored diagrams using the system.

2.4.8 User Friendly Interface

31. The system shall provide a comprehensive web-based platform that integrates all modules into a single application.
32. The system shall offer a minimalistic and easy-to-navigate interface, ensuring users can access features without complex navigation.
33. The system shall provide real-time feedback and results (e.g., diagram generation) in response to user input without requiring additional user actions.

2.5 Non-Functional Requirements

The system's nonfunctional needs are listed in this section. These requirements provide the detail of how the system behaves and responds i.e., operates. Following are the non-functional requirements:

2.5.1 Reliability

REL-1: The system must have a minimum 500-hour mean time between failures, or MTBF.

REL-2: In case of system failure, the system shall reset in under 30 seconds while keeping the data saved.

REL-3: In case of failure in diagram generation, the system shall inform user properly and provide with options such as retry.

REL-4: In case of system failure during diagram generation, the system shall restore the last stable state.

2.5.2 Usability

USE-1: With just three clicks, the user will be able to create a diagram using the system.

USE-2: Every user will be able to easily use the system's user-friendly interface.

USE-3: In order to decrease user mistakes by thirty percent, the system has to offer instructions on how to utilize the tools.

2.5.3 Performance

PER-1: Within ten seconds, the system must be able to parse the textual input and produce a diagram.

PER-2: The system must be able to process at least 50 users' queries at once without experiencing any lag.

PER-3: The system should respond within 5 seconds of incorporating some edits in the diagram.

PER-4: The system should respond within 5 seconds of optimization request in the diagram.

2.5.4 Security

SEC-1: To protect the system and previous sessions from unauthorized users, authentication will be required by the system.

SEC-2: The system shall provide protection for the personal and session data up to 90 percent.

2.6 Operation Contracts

2.6.1 Contract C-1:

Operation: Input Textual Requirements

Responsibility: Process and store textual requirements input by the user. Ensure ambiguities are resolved.

Type: System

Pre-Conditions:

- There is a user account on the system.
- The system is operational and ready to accept input.

Post-Conditions:

- Textual requirements are processed and stored.
- Ambiguities in the textual requirements are identified and resolved.

2.6.2 Contract C-2:

Operation: Generate Diagram

Responsibility: Convert textual requirements into a UML diagram and display it.

Type: System

Pre-Conditions:

- The user has inputted valid textual requirements.
- The system has processed the input text.

Post-Conditions:

- The system generates the UML diagram.
- The generated diagram is displayed on the screen.

2.6.3 Contract C-2.1:

Operation: Generate Multiple Diagrams

Responsibility: Generate different types of diagrams based on the user's input.

Type: System

Pre-Conditions:

- The user has provided valid textual requirements.
- The system supports generating multiple diagram types.

Post-Conditions:

- The system generates the requested diagram type (e.g., Class, Use Case, Sequence, or ERD).
- The diagram is displayed on the screen.

2.6.4 Contract C-3:

Operation: Optimize Diagram with AI

Responsibility: Apply AI-driven optimization to improve the diagram's layout and structure.

Type: System

Pre-Conditions:

- The user has already generated a diagram.
- The system has access to the previous diagram context.

Post-Conditions:

- The diagram is optimized based on software design standards.
- Incremental changes are applied without losing previous modifications.

2.6.5 Contract C-4:

Operation: Edit Diagram in DSL Editor

Responsibility: Allow the user to manually edit the diagram using the DSL editor.

Type: System

Pre-Conditions:

- The user has access to a generated diagram.
- The user is familiar with DSL syntax (e.g., PlantUML).

Post-Conditions:

- The diagram is updated in real-time based on the changes made.
- The changes are saved for future use or export.

2.6.6 Contract C-5

Operation: Customize Diagram with Themes

Responsibility: Allow the user to apply themes and color palettes to the diagram for customization.

Type: System

Pre-Conditions:

- The user has a generated diagram available for customization.
- The system provides available themes and color options.

Post-Conditions:

- The diagram is customized according to the selected theme or color palette.
- The customized diagram is rendered in real-time and saved.

2.6.7 Contract C-6:

Operation: Export Diagram in Multiple Formats

Responsibility: Export the generated diagram in different formats (e.g., SVG, PNG).

Type: System

Pre-Conditions:

- The user has successfully generated a diagram.
- The system provides export options for multiple formats.

Post-Conditions:

- The diagram is exported in the selected format (e.g., SVG, PNG).
- The user is able to download the exported diagram.

2.6.8 Contract C-7:

Operation: Save Diagram Code and Session

Responsibility: Save the diagram code and workspace session for future access.

Type: System

Pre-Conditions:

- There is a user account on the system.
- The user has created or edited a diagram in the current session.

Post-Conditions:

- The diagram code is saved for future retrieval.
- The user's workspace and session data are stored for future access.

2.7 Domain Model

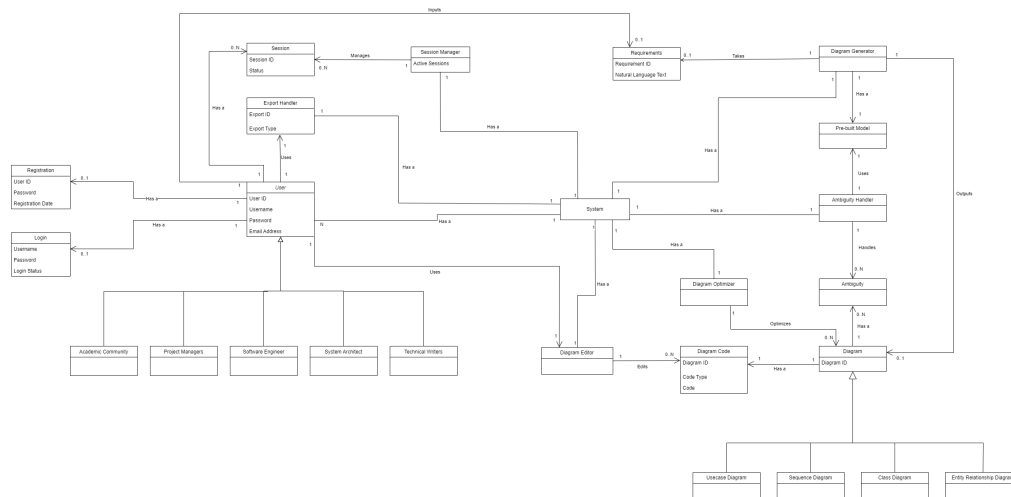


Figure 2.2: Domain Model

Chapter 3

System Overview

AiUML is a system developed to automatically create UML diagrams from natural language requirements. Its primary function is to receive user input in the form of text, find any ambiguities in the requirements, and then produce precise and understandable UML diagrams, such as class, use case, sequence, and entity-relationship diagrams. AiUML helps simplify the process of designing object-oriented software by converting complex user requirements into easy-to-understand visual diagrams.

The design methodology for this system is **Procedural**. The related design artifacts are given below.

3.1 Architectural Design

AiUML follows **Client-Server Architecture** consisting of a client-side web interface and a server on the back-end which is hosting multiple services.

Client Side:

- On the client side, users interact with the system via a simple web interface where they input textual requirements, customize diagrams, and view the generated outputs.

Server Side:

- The back-end Server handles text processing, ambiguity detection, and diagram generation using large language models.
- The server also has services for diagram optimization using pre-trained machine learning models, customization, editing and exporting diagrams.

Storage:

- The system stores diagrams, requirements, users and session data in databases for future access and modification.
- The database is depicted separately from the server components but logically part of the server-side.

3.2 Architecture Diagram

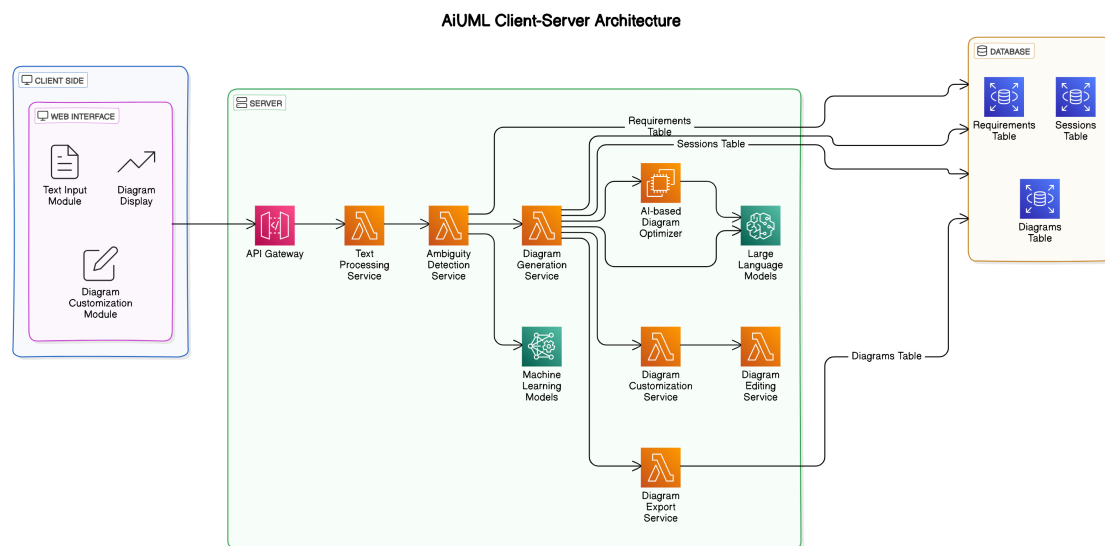


Figure 3.1: Architecture Diagram

3.3 Activity Diagrams

The following activity diagrams explain the interactions between users and the system during particular actions, showing the sequential procedures involved in various use cases. Each diagram captures both normal and alternative flows to provide a comprehensive view of system behavior.

3.3.1 AD -1: Input Textual Requirements

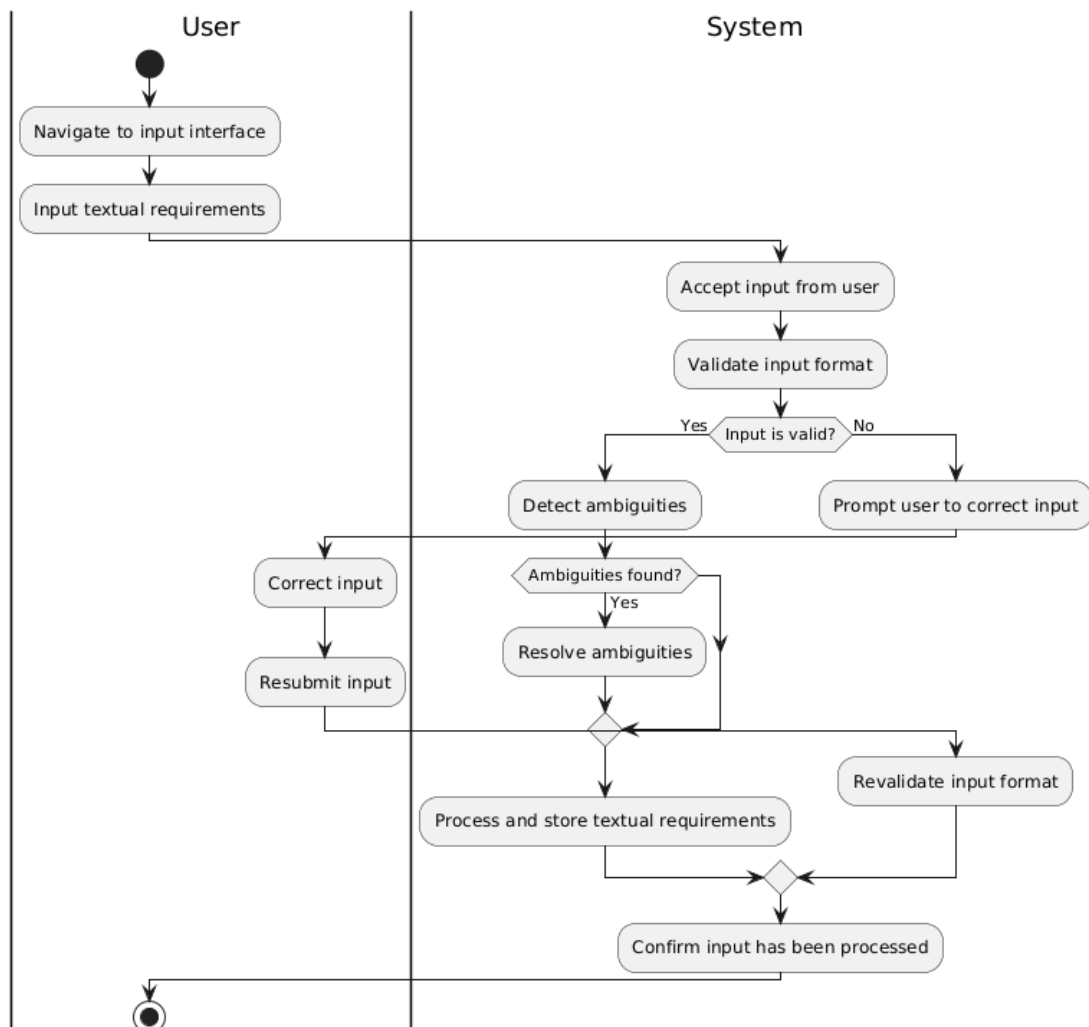


Figure 3.2: AD1 Input Textual Requirements

3.3.2 AD -2: Generate Diagram

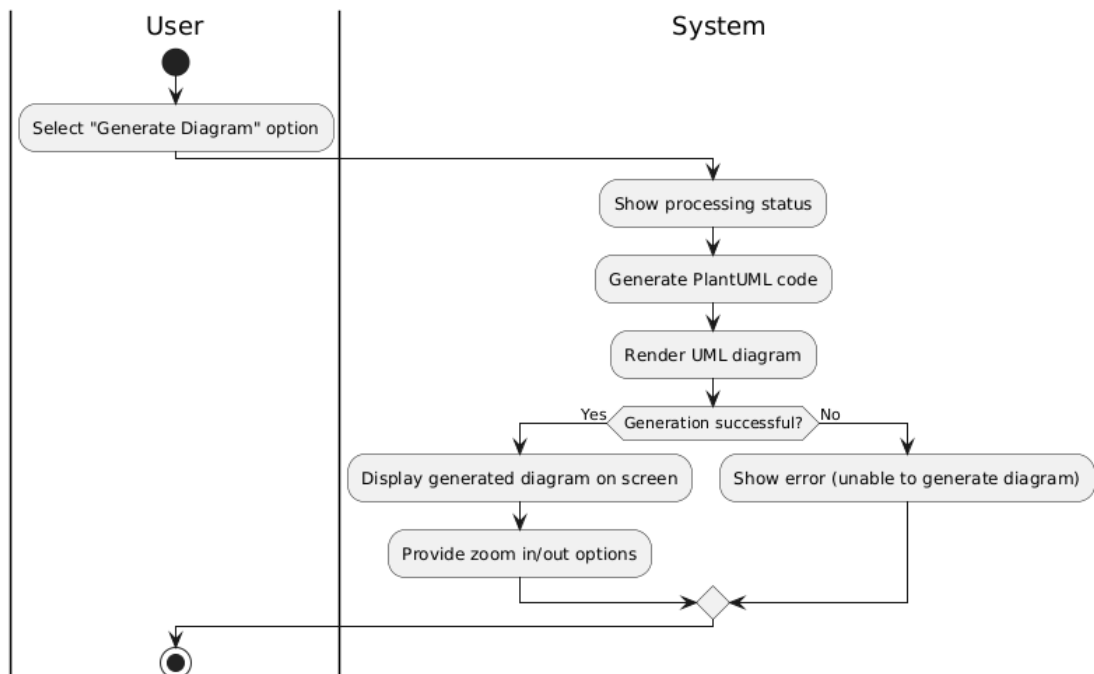


Figure 3.3: AD2 Generate Diagram

3.3.3 AD -3: Optimize Diagram with AI

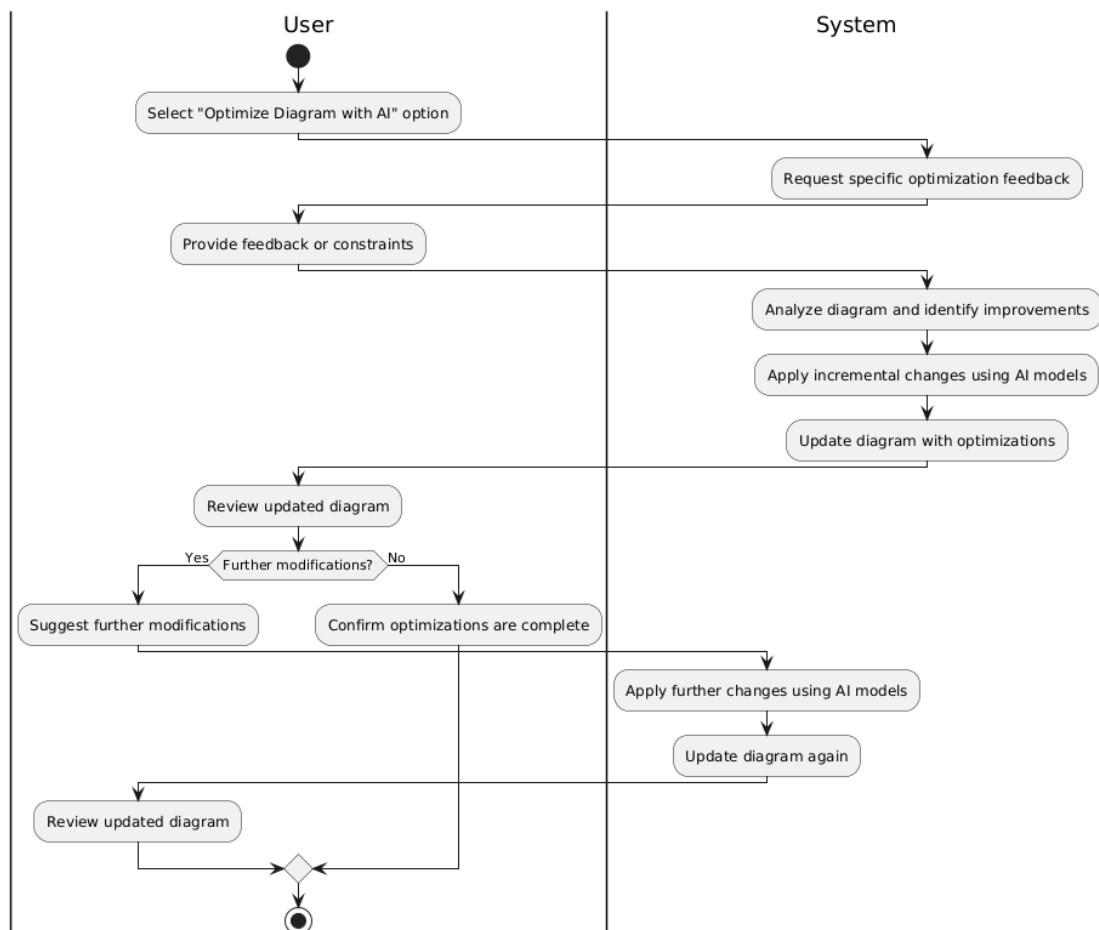


Figure 3.4: AD3 Optimize Diagram with AI

3.3.4 AD -4: Edit Diagram in DSL Editor

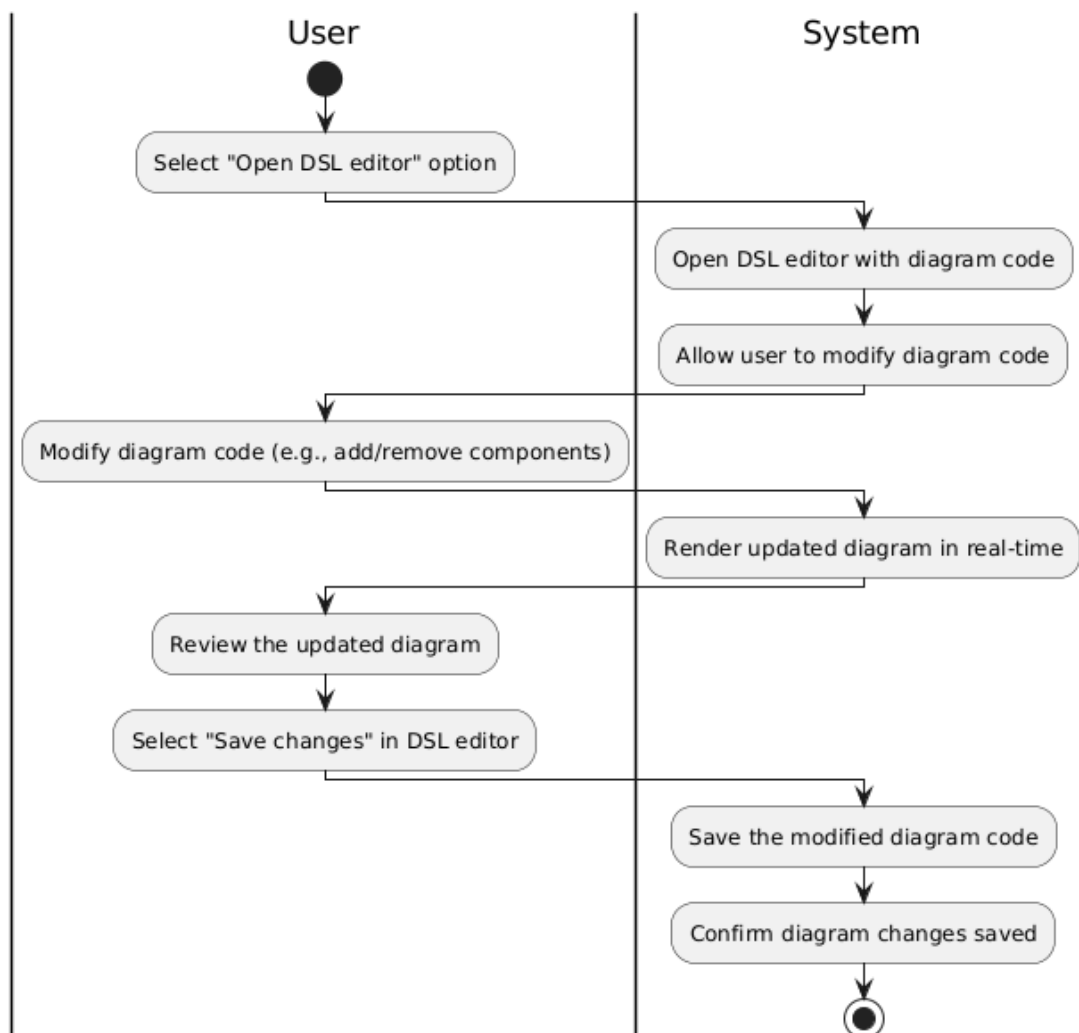


Figure 3.5: AD4 Edit Diagram in DSL Editor

3.3.5 AD -5: Customize Diagram with Themes

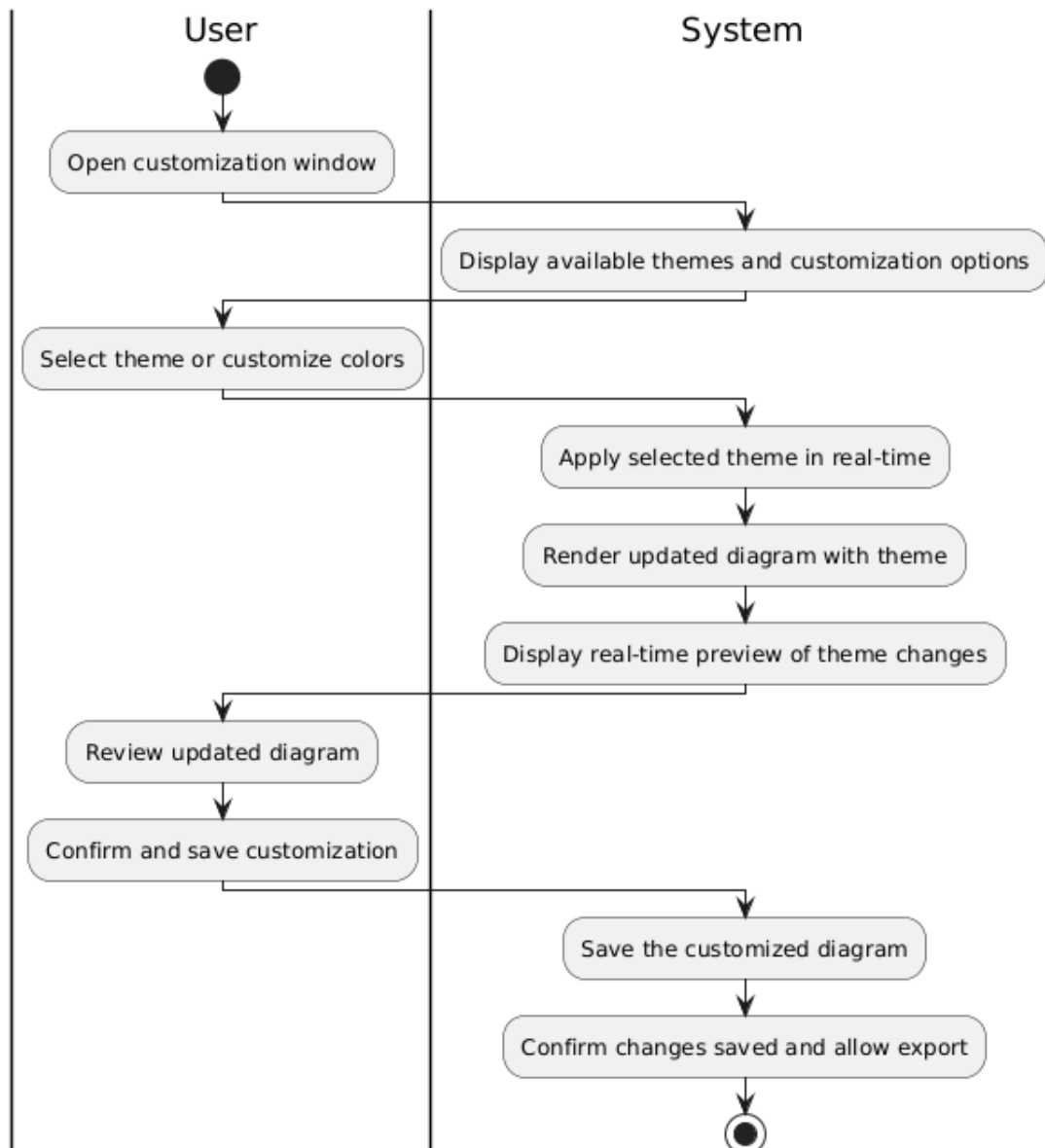


Figure 3.6: AD5 Customize Diagram with Themes

3.3.6 AD -6: Export Diagram in Multiple Formats

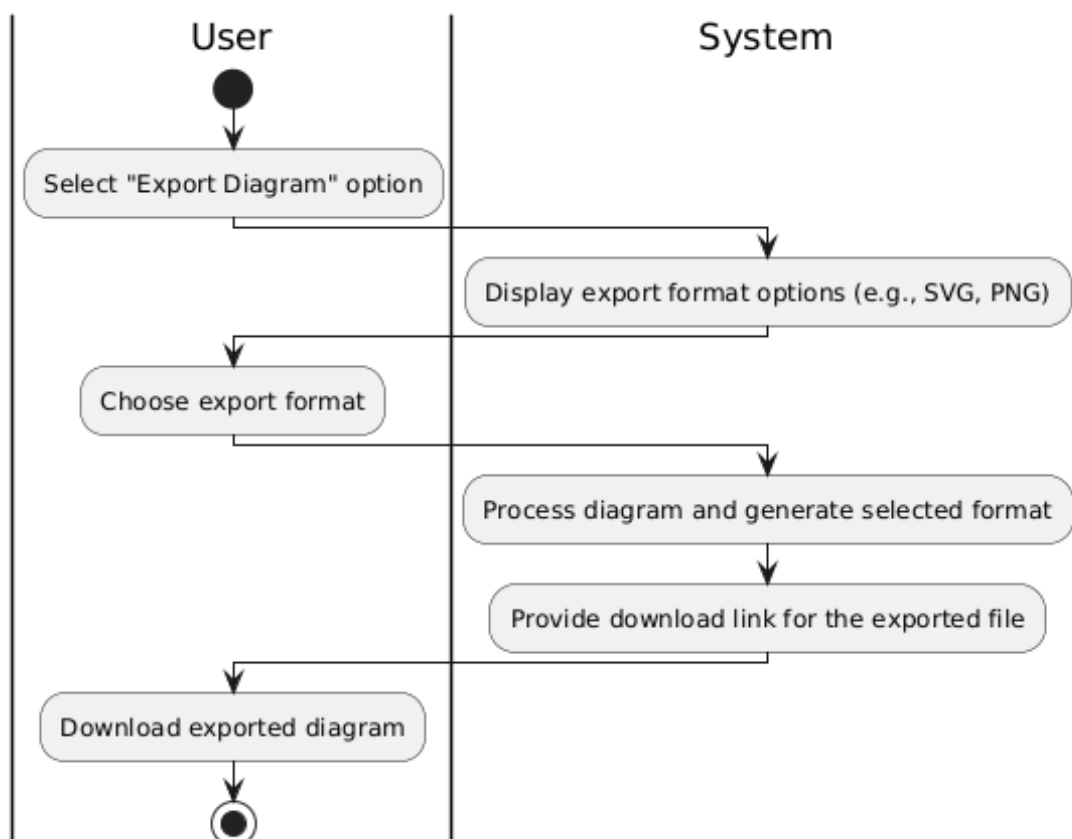


Figure 3.7: AD6 Export Diagram in Multiple Formats

3.3.7 AD -7: Session and Diagram Code Storage

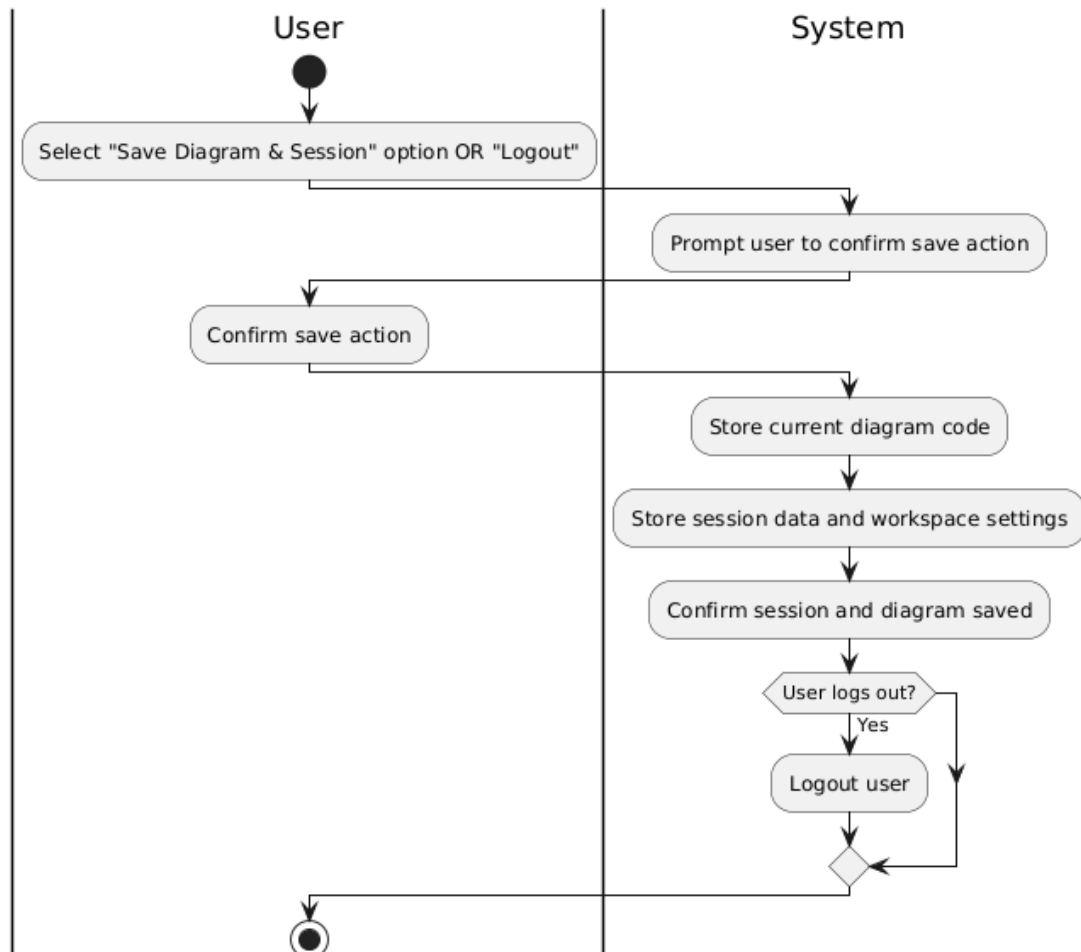


Figure 3.8: AD7 Session & Diagram Code Storage

3.4 State Transition Diagram

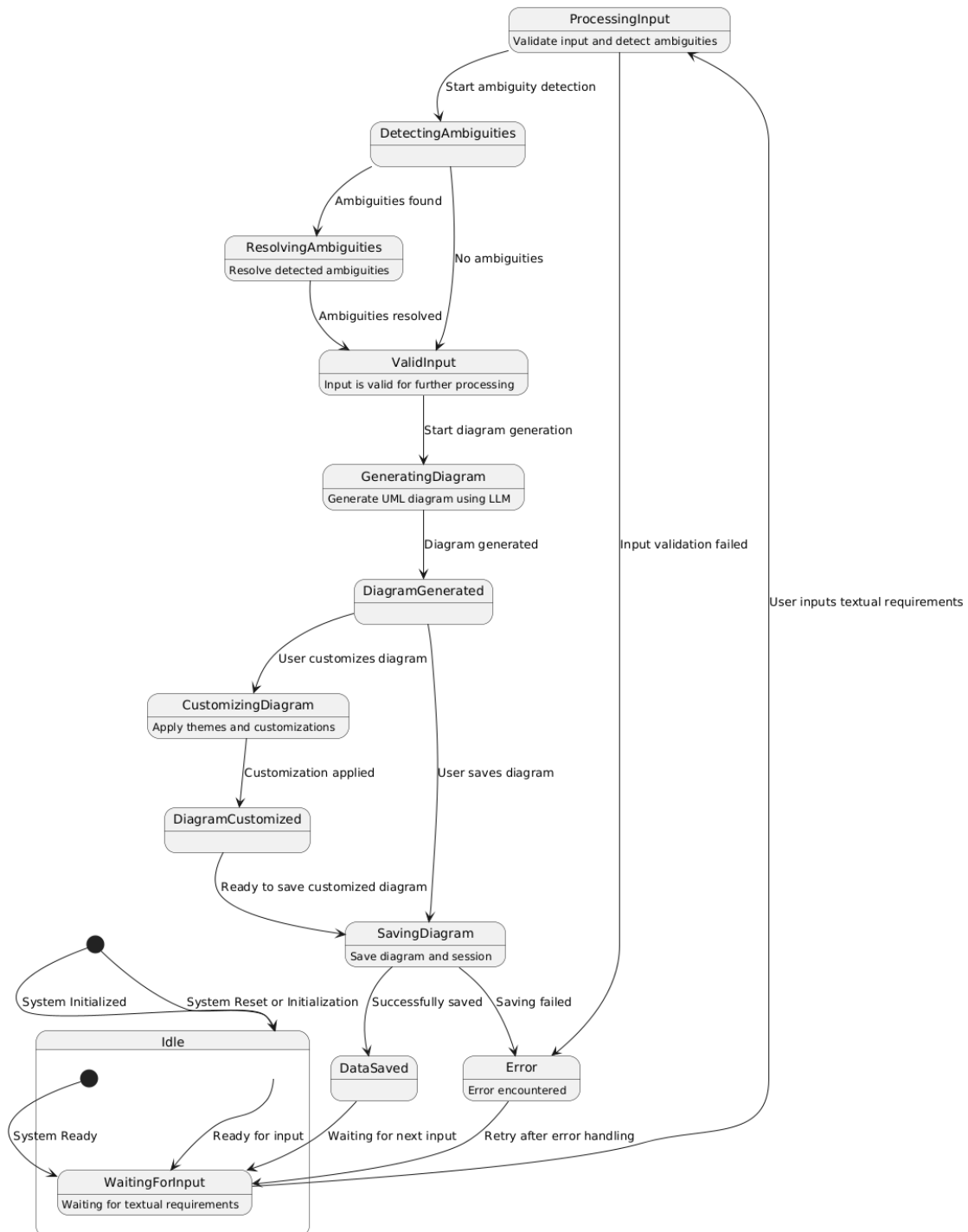


Figure 3.9: State Transition Diagram

3.5 Data Flow Diagram

The procedures, data storage, and information flow across the various modules of the system are depicted in this Data Flow Diagram (DFD), which shows how textual requirements are processed, optimized, and customized UML diagrams are created, and then exported or shown to the user.

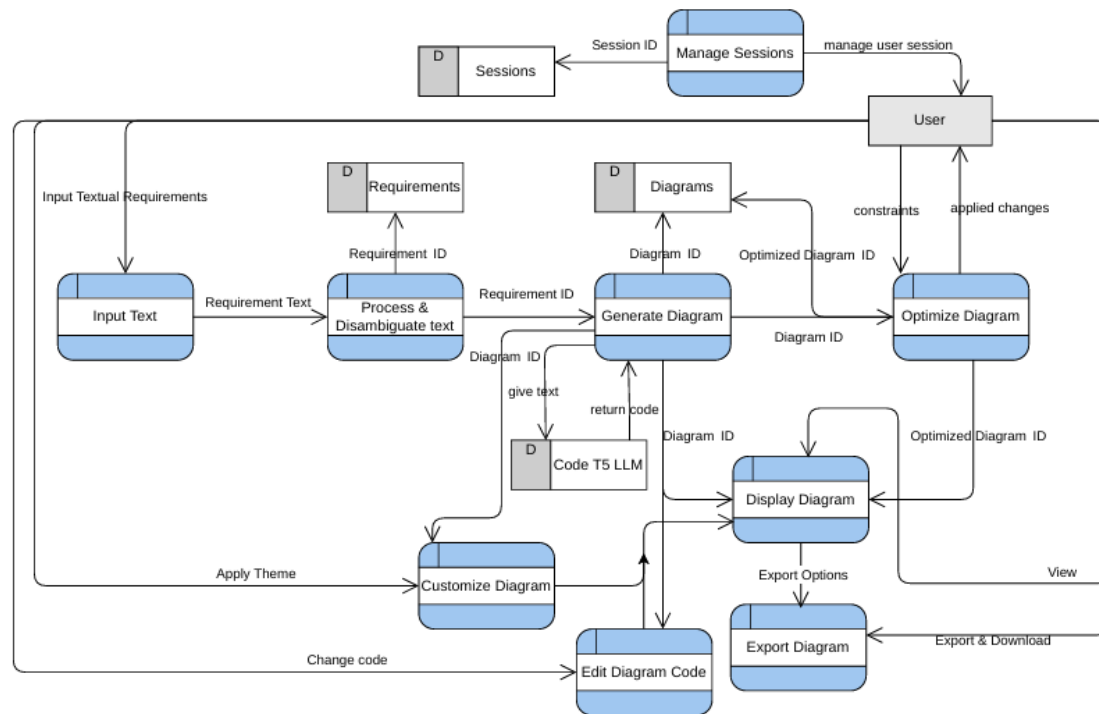


Figure 3.10: Data Flow Diagram

3.5.0.1 Processes (Blue Color):

1. Input Text:

- **Source:** User.
- **Description:** This process receives input textual requirements from the user.
- **Destination:** Sends the text to the "Process & Disambiguate Text" process.

2. Process & Disambiguate Text:

- **Source:** "Input Text" process.

- **Description:** It processes the user's textual requirements and resolves any ambiguities.
- **Destination:** Sends the processed requirement text to the "Generate Diagram" process. Stores the processed requirements in the **Requirements** data store.

3. Generate Diagram:

- **Source:** "Process & Disambiguate Text".
- **Description:** Generates the UML diagram based on the processed requirements. It communicates with the **CodeT5 LLM** to generate the PlantUML code.
- **Destination:** Stores the generated diagram in the **Diagrams** data store and sends the diagram to the "Optimize Diagram" process.

4. Optimize Diagram:

- **Source:** "Generate Diagram".
- **Description:** Optimizes the generated diagram based on AI-driven adjustments. Takes constraints and applied changes from the user.
- **Destination:** Sends the optimized diagram to both the "Display Diagram" and "Export Diagram" processes.

5. Display Diagram:

- **Source:** "Optimize Diagram".
- **Description:** Displays the optimized diagram to the user for viewing.
- **Destination:** Sends the displayed diagram back to the user for viewing.

6. Customize Diagram:

- **Source:** User.
- **Description:** Allows the user to apply themes or customize the appearance of the diagram. It modifies the appearance of the diagram and applies themes.
- **Destination:** Sends the customized diagram to the "Display Diagram" and "Export Diagram" processes.

7. Edit Diagram Code:

- **Source:** User.
- **Description:** Allows the user to modify the diagram code directly. The changes applied by the user are reflected in the diagram.

- **Destination:** Sends the modified diagram to the **Diagrams** data store.

8. Export Diagram:

- **Source:** "Optimize Diagram" or "Customize Diagram".
- **Description:** Exports the customized or optimized diagram in various formats (e.g., PNG, SVG).
- **Destination:** Sends the exported diagram back to the user for downloading.

9. Manage Sessions:

- **Source:** User.
- **Description:** Manages user sessions, including session creation, storage, and retrieval.
- **Destination:** Stores session data in the **Sessions** data store.

3.5.0.2 Data Stores (Gray Color):

1. Requirements DB Table:

- **Data:** Stores processed textual requirements received from the user.
- **Processes:** Receives data from "Process & Disambiguate Text".

2. Diagrams DB Table:

- **Data:** Stores generated and customized UML diagrams.
- **Processes:** Receives data from "Generate Diagram" and "Edit Diagram Code". Updates are made by the "Customize Diagram" process.

3. Sessions DB Table:

- **Data:** Stores session data for user interactions.
- **Processes:** Receives data from "Manage Sessions".

4. Code T5 LLM:

- **Data:** Used for generating PlantUML code from textual requirements.
- **Processes:** Communicates with the "Generate Diagram" process.

3.5.0.3 External Entities:

User: The primary source for input textual requirements, user constraints, and applied changes. The user interacts with the system to view, edit, or download diagrams. The flow of data across the processes ensures that user inputs are transformed into useful diagrams, and optimized and customized results are made available for viewing, exporting, or further editing.

3.6 System Sequence Diagrams

The high-level use cases of the AiUML system's system-level sequence diagrams illustrate how various parts work together to accomplish activities like importing requirements, creating diagrams, using AI to optimize them, and exporting them. Each system sequence diagram shows the flow of data between components, making it clear how the system operates end-to-end.

3.6.1 SSD -1: Input Textual Requirements

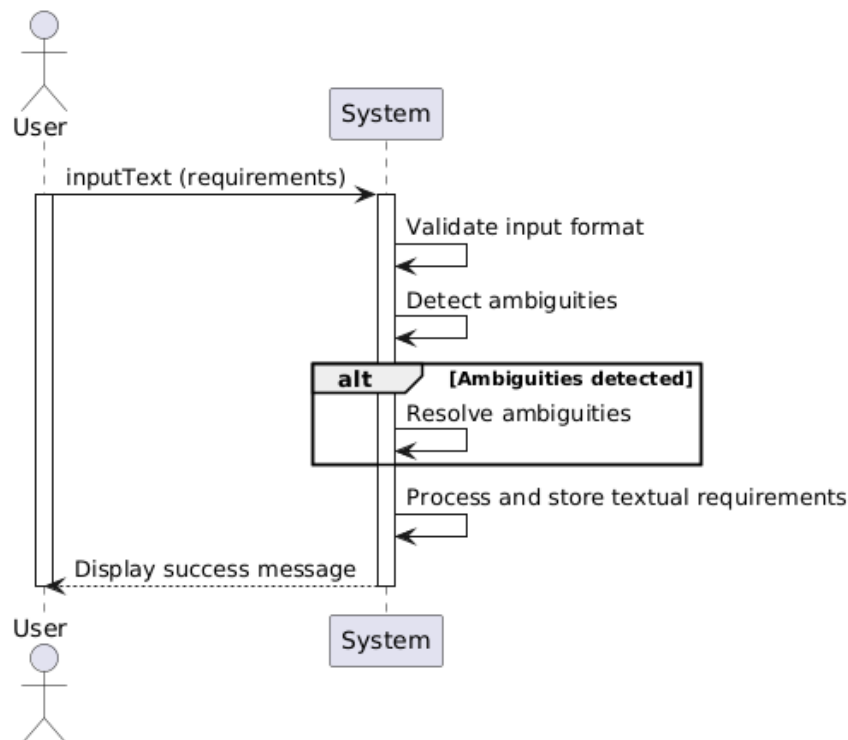


Figure 3.11: SSD1 Input Textual Requirements

3.6.2 SSD -2: Generate Diagram

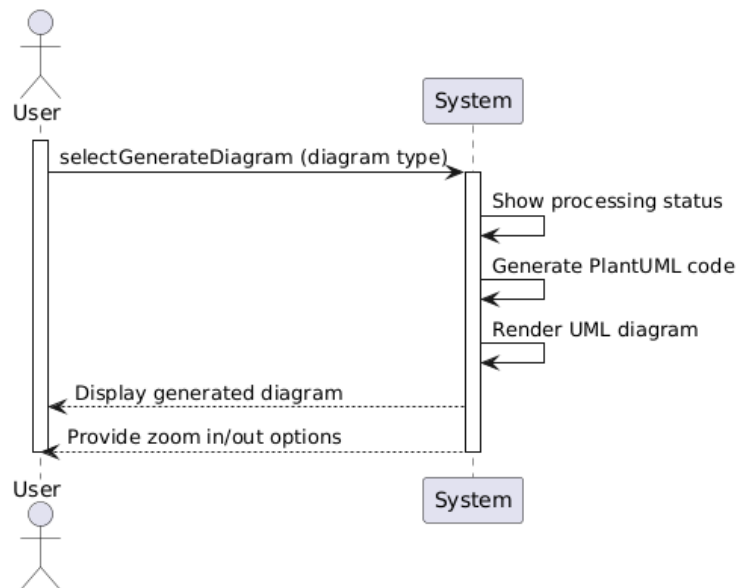


Figure 3.12: SSD2 Generate Diagram

3.6.3 SSD -3: Optimize Diagram with AI

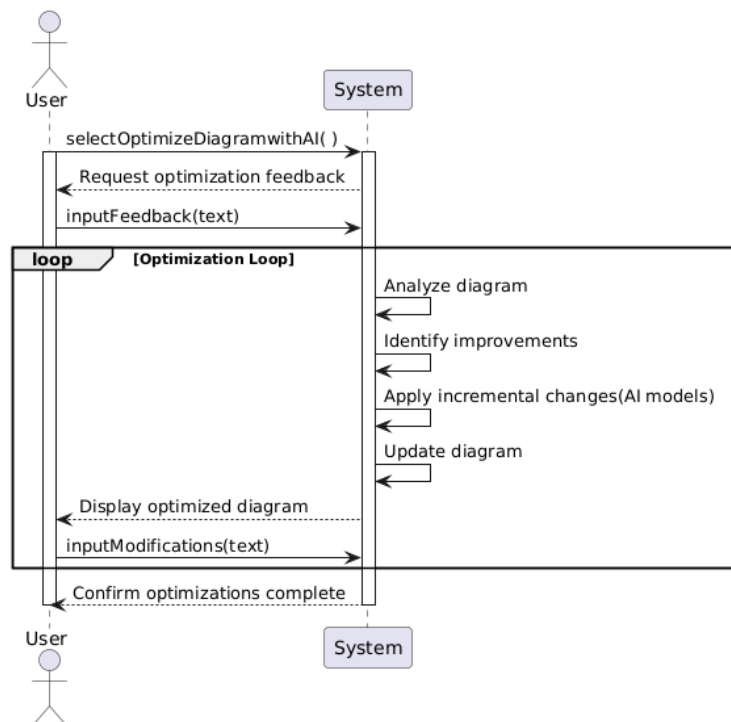


Figure 3.13: SSD3 Optimize Diagram with AI

3.6.4 SSD -4: Edit Diagram in DSL Editor

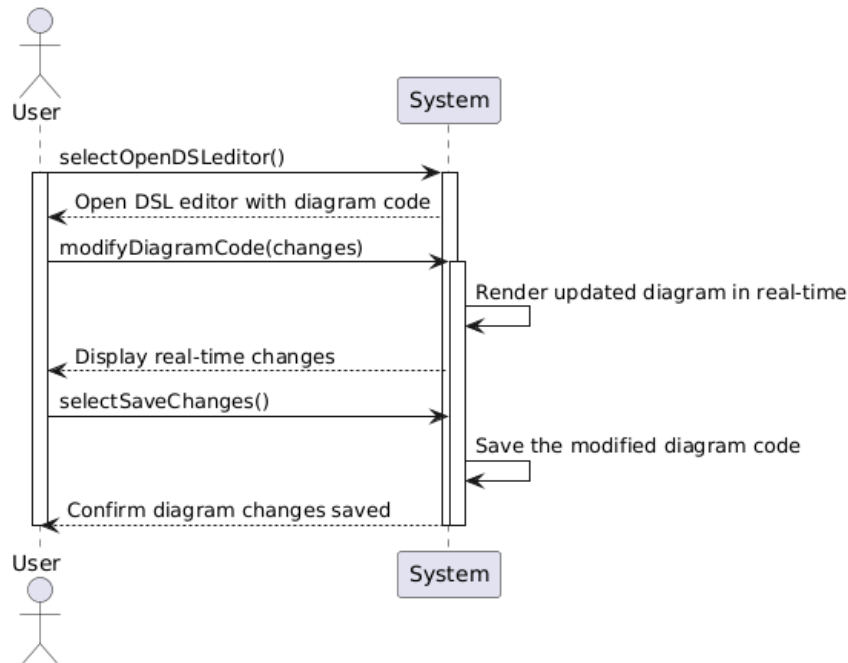


Figure 3.14: SSD4 Edit Diagram in DSL Editor

3.6.5 SSD -5: Customize Diagram with Themes

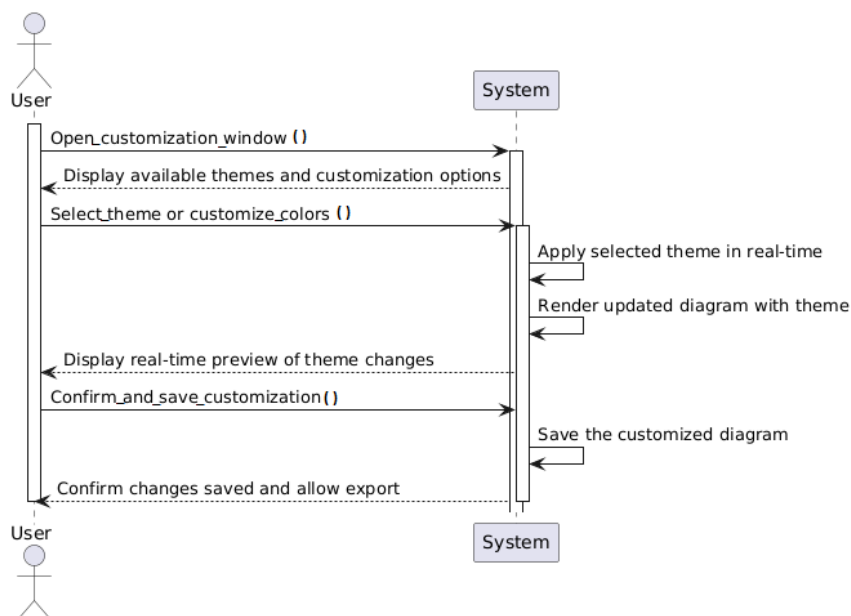


Figure 3.15: SSD5 Customize Diagram with Themes

3.6.6 SSD -6: Export Diagram in multiple formats

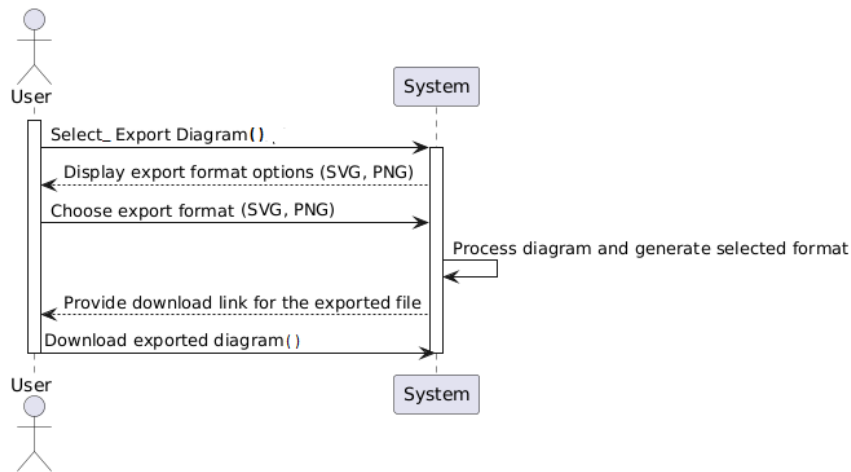


Figure 3.16: SSD6 Export Diagram in Multiple Formats

3.6.7 SSD -7: Session and Diagram Code Storage

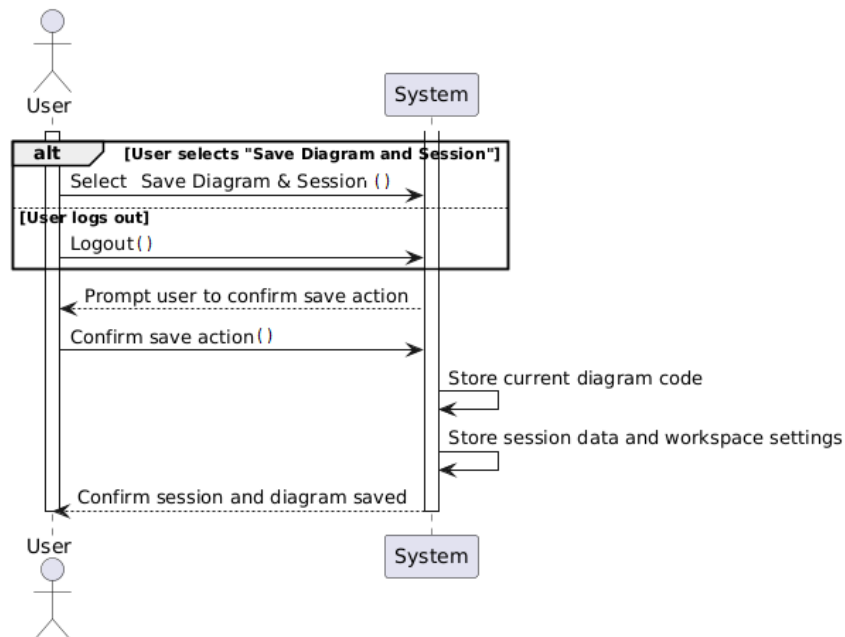


Figure 3.17: SSD7 Session and Diagram Code Storage

3.7 Data Design

AiUML system transforms its information domain into structured data entities, which are stored and managed in a Database. This database consists of key entities such as User, Session, Requirements, and Diagram, each corresponding to a table that supports the system's functionality.

3.7.1 Key Entities:

All these entities are interconnected and processed by the back-end services. The database items include:

- User Table
- Session Table
- Requirements Table & Optimization Requests Table
- Diagrams Table & Diagram Code Table
- Ambiguities Table
- Themes Table (pre-defined)

For example, the User table stores user information like UserID, Username, and Password, while the Requirements table holds the natural language text that users input for diagram generation. The Session table tracks active user sessions and allows the system to retrieve and manage ongoing activities. The Diagram table stores generated UML diagrams, their metadata, and code, which can later be accessed for editing or export.

3.7.2 Storage and Processing:

The Database is the central storage for AiUML, with specific entities (tables) that store data related to users, sessions, requirements, diagrams, and ambiguities. Each entity in the ERD corresponds to a table in the database, and the relationships between these entities are reflected in how the data is processed:

- Requirements are processed by the Text Processing Service and Ambiguity Detection Service to generate accurate UML diagrams.
- Sessions are maintained to track user activities and allow for session-based diagram retrieval and editing.

- Diagrams are stored and managed through the Diagram Generation Service, which also integrates with the AI-based Diagram Optimizer for refinement.

This structure facilitates smooth data processing, retrieval, and storage, and thus supports the overall functionality of AiUML system.

3.8 Entity Relationship Diagram

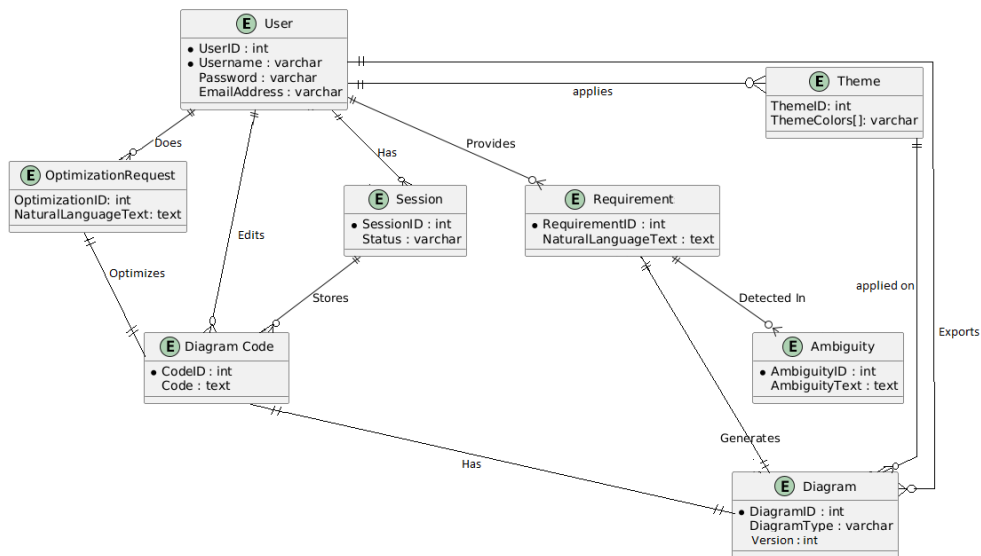


Figure 3.18: Entity Relationship Diagram (UML notation)

Bibliography

- [1] Abdelkareem M. Alashqar. Automatic generation of uml diagrams from scenario-based user requirements. *Jordanian Journal of Computers and Information Technology* 7(2):1 7(2):1, 2021.
- [2] Zakaria Babaalla, Hamza Abdelmalek, Abdeslam Jakimi, and Mohamed Oualla. Extraction of uml class diagrams using deep learning: Comparative study and critical analysis. *Procedia Computer Science*, 236:452–459, 2024.
- [3] Zakaria Babaalla, El Mahi Bouziane, Abdeslam Jakimi, and Moahmed Oualla. From text-based system specifications to uml diagrams: A bridge between words and models. In *2024 International Conference on Circuit, Systems and Communication (ICCSC)*, pages 1–6, 2024.
- [4] Zakaria Babaalla, Abdeslam Jakimi, Mohamed Oualla, Rachid Saadane, and Abdelilah Chehri. Towards an automatic extracting uml class diagram from system’s textual specification. In *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security (NISS ’24)*, pages 1–5, New York, NY, USA, 2024. Association for Computing Machinery.
- [5] Souvick Das, Novarun Deb, Agostino Cortesi, and Nabendu Chaki. Zero-shot learning for named entity recognition in software specification documents. In *2023 IEEE 31st International Requirements Engineering Conference (RE)*, pages 100–110, 2023.
- [6] Yasitha Gamage. *Automated Software Architecture Diagram Generator using Natural Language Processing*. PhD thesis, 05 2023.
- [7] Song Yang and Houari Sahraoui. Towards automatically extracting uml class diagrams from natural language specifications. In *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, MODELS ’22*, page 396–403, New York, NY, USA, 2022. Association for Computing Machinery.
- [8] Shaohong Zhong, Andrea Scarinci, and Alice Cicirello. Natural language processing for systems engineering: Automatic generation of systems modelling language diagrams. *Knowledge-Based Systems*, 259:110071, 2023.